Introduction to DNS

Introduction

- In this lesson we will cover the basics of configuring a DNS server for your network
  - Basic DNS functionality
  - Important terms
  - Configuration options
  - Zone files

DNS Refresher

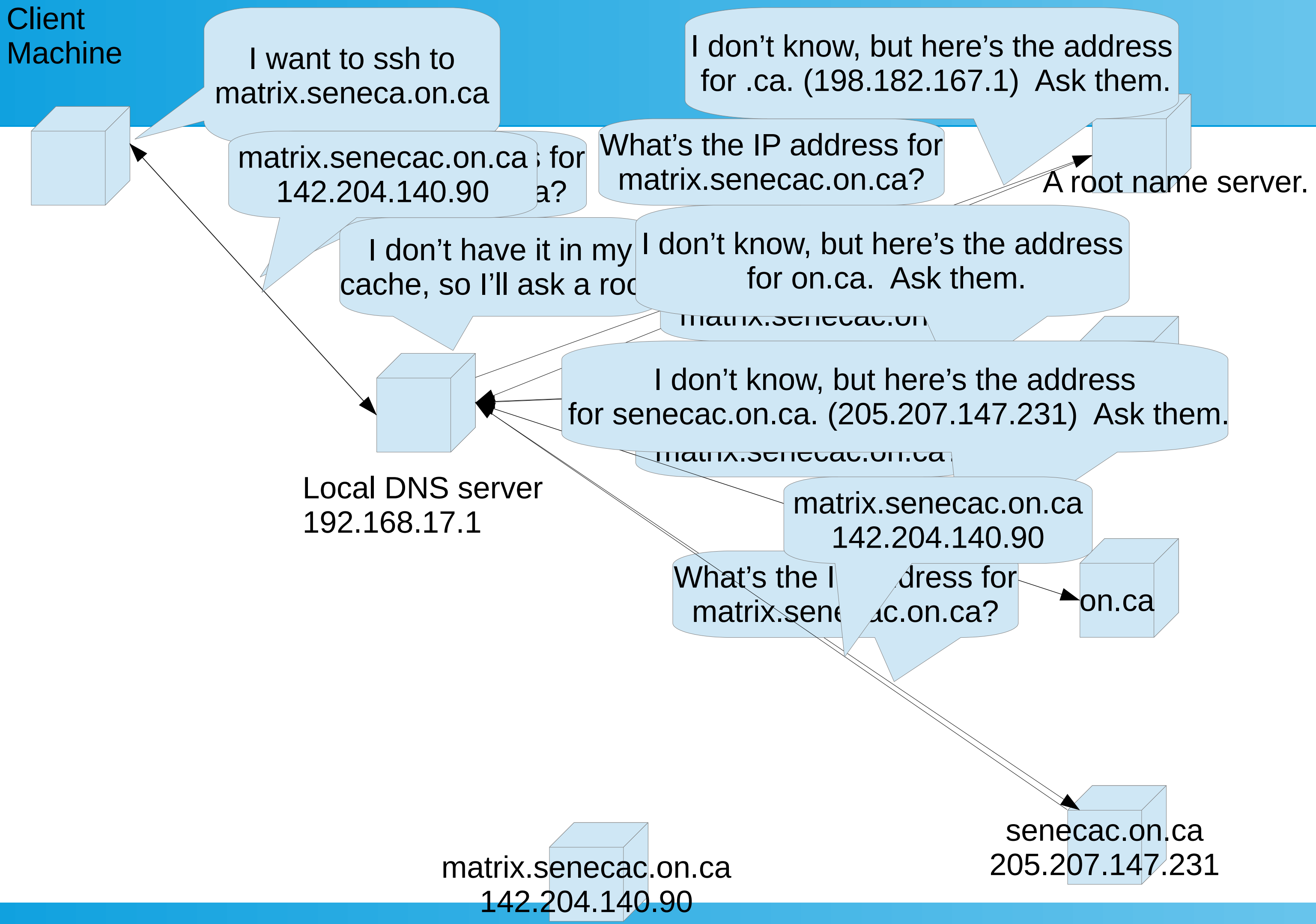
- When two machines want to communicate over the network, they need to know where each other are.
  - An IP address
- Most of the time, users don't deal with IP addresses.
  - Names are much easier to remember
    - Does anyone know the IP of the seneca email server?

DNS Refresher

- Something needs to translate between the human-readable names and the machine-useful IP addresses (forward mapping).
  - You can do this with a static file (/etc/hosts), but you would need one of those for each machine, would need to edit them all every time anything changed, and would need to know the IP address to begin with.
    - This could work locally, for small networks, but there is no way the internet could work like this.
- Something also needs to translate the other way (reverse mapping).
  - When you have an IP and need to know the name associated with it.

DNS Refresher

- Instead, there is an hierarchical, distributed database that divides machines up into zones (similar to domains).
- When you are trying to find a machine, you ask a name server for that zone.
  - Don't know where the name server is?
    - Ask the name server for its zone.
- e.g. trying to find matrix.senecac.on.ca


Client Machine
I want to ssh to matrix.seneca.on.ca
I don't know, but here's the address for .ca. (198.182.167.1) Ask them.
What's the IP address for matrix.senecac.on.ca?
A root name server.
matrix.senecac.on.ca 142.204.140.90
I don't know, but here's the address for on.ca. Ask them.
I don't know, but here's the address for senecac.on.ca. (205.207.147.231) Ask them.
Local DNS server 192.168.17.1
matrix.senecac.on.ca 142.204.140.90
on.ca
senecac.on.ca 205.207.147.231
matrix.senecac.on.ca 142.204.140.90

DNS Refresher

- For all of this to work, there has to be somewhere where you can go if you don't know anything.
- These are root name servers.
  - They are in known locations (and don't change) and know the locations of the name servers for the top level domains.
    - .edu, .gov, .com, .net, .ca, etc.
    - Each of those will know about the name servers in that zone. And those will know about the ones below them.
  - Remember, domains are actually read right to left (by machines, anyway).

Terminology

- Zone vs Domain:
  - A domain is a named section of a network (e.g. on.ca) and everything inside (below) it (e.g. senecac.on.ca).
  - A zone is the domain, without any of its sub-domains.
    - Just on.ca, not including senecac, or anything else.
  - Name servers are responsible usually for a zone. They will (should) know about the name servers that represent zones in sub-domains.
    - But not the other machines
    - This is what creates that hierarchical structure.

Terminology

- Primary and Secondary
  - AKA Master and Slave
  - A primary server holds a copy of the answers it can provide on the local disk.
  - An admin can manually change these.
  - Secondary get copies of the data from the primary.
    - And must not be manually changed.
  - This spreads the load across multiple machines and protects you in case one machine fails.
  - Any zone must have at least one primary, and should have more than one secondary.
    - One of which should be off-site.

Installation

- We'll be using a service called named (name-dee). The package to install can vary between distributions but is typically one of named, bind, or bind9.
  - On ubuntu, we'll use bind9.
- Most configuration will take place in named.conf
  - Some distributions store this as one big file, some split it into several smaller ones.
  - Ubuntu stores it in /etc/bind, and splits it into several files.
    - We'll be interested in named.conf.options and named.conf.local

Named.conf.options

- This file contains configuration parameters that control how the server will behave.
- E.g. who it will respond to, what kind of responses it can give, what other servers it will talk to.

Configuring DNS - options

- The options statement allows you to set a number (~170) of variables that modify how the server will behave.
- There are far too many to list here.
  - I've selected a small handful that have significant effects on how the server functions.
    - We will see more over the next few lectures.
- Most of these can be set globally (in an options statement), or for particular zones (in each zone statement).

Address Lists

- Several options take address lists as values.
- These are groups of addresses (host or network).
- { address; address; address; };
- e.g.
- { 142.204.27/24; 142.204.133/24; };
  - Note the slightly modified CIDR.

Controlling Recursion

- recursion {yes|no} – whether this server is recursive or not. Note that some client resolver software does not understand referrals, so if this is a server your clients will ask (instead of just other DNS servers), it must be recursive.
- allow-recursion {address list} – This will limit the hosts this machine will be recursive for to just those in the list.
  - This is a handy way to make it recursive for your own clients, and non-recursive for everyone else.

Controlling Forwarding

- forwarders {address list} – other DNS servers to query first, before resorting to external queries.
  - This does not affect reverse lookups.
- forward {only|first} – sets whether this server will query its listed forwarders before going external (first), or will only query the forwarders and give up if they do not have the answer (only).

Who Can Query

- allow-query {address list} – limit the machines that can query this server to only those in the address list.
- allow-query-cache {address list} - limit the machines that can query this server's cache to only those in the address list.

Zone Entries

- Each zone entry (held in named.conf.local) defines one zone that this server will be a name server for.
- At minimum:

```
zone “domain name” {
  type primary|secondary|forward|stub;
  file <path to zone file>;
};
```

- This can also include the options described earlier (in which case they affect only this zone).

Zones

- Inside the zone files is where the actual data is stored.
    - There are several directories these could be stored in, but use /var/cache/bind.
- Each entry in a zone file(a resource record) represents a piece of data (i.e. a host, a service, etc).
- There are many types of resource records, so we will again be limited to common (basic) ones.

Configuring DNS - Zones

- $TTL (seconds) – sets the default Time-To-Live for entries listed in this zone. This determines how long cached copies of this data are considered valid.
  - You can include this in every record, but it is more commonly set once as a default for all. And maybe explicitly changed on unique or special records.
  - Instead of seconds you can also append m (minutes), h (hours), d (days), or w (weeks).
    - This applies to any TTL value.
    - Also accepts upper case.

Resource Records

- The most common resource records you will deal with are A (for address), and PTR (for pointer).
- Each zone also needs an SOA (Start Of Authority) record for the zone.
  - We will look at these in detail later.
- You will also need NS records to locate Name Servers.
- You might also include TXT (text) records, or MX (Mail eXchange).
  - We'll see MX and TXT records in OPS300 when we deal with mail servers.

A Records

- A describes a forward mapping entry (I have a name and need the matching IP address).
- *hostname TTL type* A *address*
  - You can leave out the *TTL* value if you have set up a default in this zone file.

```
matrix.senecac.on.ca.    2W       IN       A         142.204.140.90
```

- Note the . on the end. It represents the internet's 'root'. If you leave that off, the hostname is treated as relative and the value you set in $ORIGIN for this zone file is appended, or the zone statement if you did not define $ORIGIN.

Configuring DNS - Zones

- NS records identify name servers in this zone.
- *zone TTL IN* NS *hostname*
- If *zone* is left out, it uses the same zone as the resource record preceding it.
- NS records are also often listed immediately after the SOA record, so you will usually see them without ZONE (and TTL).

```
senecac.on.ca. 1w IN NS ns1.senecac.on.ca.
```

- Or

```
        IN    NS    ns1.senecac.on.ca.
```

Aside: Blanks

- If you leave a space at the beginning of a line, the server will assume that this record applies to the same machine/domain as the previous record.
- Very handy if you have several records for one machine.
- Also a very common cause of errors.

Reverse DNS

- The zone you already set up will only handle forward lookups
  - Clients have a hostname and are searching for the ip address.
- You need more to handle reverse lookups
  - Clients have an ip address and need the associated hostname.
  - If you have a forward zone, you will almost always need the matching reverse zone.

Reverse Zone Names

- In order to make DNS software simpler, the original designers made ip addresses act like named zones, with each octet acting like a sub-domain.
  - e.g. 168 is a sub-domain of 192
  - All of that resides in in-addr.arpa.
- This means you need a zone to handle searches that use ip addresses.

Reverse Zone Entries

- Create a zone entry in /etc/bind/named.conf.local for your network.

```
zone “X.168.192.in-addr.arpa” {
  type primary;
  file <path to zone file>;
};
```

Reverse Zones

- In the zone file you'll need at least:
  - One SOA record (provided in the lab – we'll examine it closer next week).
  - One NS record
- A default $TTL is optional
- Then you need the actual resource records
  - The most common record in a reverse zone is PTR

PTR Records

- PTR describes a reverse mapping entry (I have an IP address and need a name).
- *address TTL* IN PTR *hostname*

```
90.140.204.142.in-addr.arpa.   2W        IN        PTR      matrix.senecac.on.ca.
```

- Note that the octets are listed backwards.
  - This is to maintain compatibility with the forward mapping software.
- If you do not end with "." it will append the octets listed in the zone statement with in-addr.arpa.
  - This and leaving out the . in A records are very common causes of errors.

Summary

- In this lesson we have briefly covered how to configure a primary DNS server for you network
    - Several Key terms
    - Common options
    - Zone entries
    - Zone files
        - A few resource record types
        - A, PTR, NS